0) Filosofi Setup (biar nggak ambigu)

Kita **lock versi** persis sesuai SRS tadi (major/minor jelas), dan hindari auto-upgrade. Patch boleh naik kalau **compatible**, tapi kita simpan di lockfile biar deterministic.

Versi yang dipakai (wajib): - **Node.js**: **20.x LTS** (pakai `.nvmrc` → `20`) - **pnpm**: **9.x** - **Next.js**: **14.2.x** (App Router, React 18) - **React / React-DOM**: **18.2.0** - **Tailwind CSS**: **3.4.x** - **TanStack Query**: **5.x** - **shadcn/ui**: sesuai generator untuk Next 14 (CLI) - **React Hook Form**: **7.x** | **Zod**: **3.x** | **Recharts**: **2.10.x+** - **NestJS**: **10.x** - **Prisma**: **5.x** | **PostgreSQL**: **16.x** - **Redis**: **7.x** | **Kafka**: **3.7.x** (KRaft, tanpa Zookeeper) - **Keycloak**: **24.x** (LTS lini 24)

> Catatan: angka **x** = patch terbaru dalam satu lini yang sama. Kita kunci dengan lockfile; kalau perlu full pin (mis. CI), set exact patch.

---

1) Prasyarat Mesin Dev

1. **Git**
2. **Node 20 LTS** + **nvm** (Linux/Mac) / **fnm** (opsional).

```
# macOS (Homebrew)
brew install nvm && mkdir -p ~/.nvm
echo 'export NVM_DIR="$HOME/.nvm"; [ -s "/opt/homebrew/opt/nvm/nvm.sh" ]
&& . "/opt/homebrew/opt/nvm/nvm.sh"' >> ~/.zshrc
nvm install 20 && nvm use 20
```

3. **pnpm 9**: `npm i -g pnpm@9`
4. **Docker Desktop** (Compose v2 aktif) / **docker + docker compose** di Linux.
5. (Opsional) **mkcert** untuk HTTPS lokal.

Buat `.nvmrc` di root repo berisi `20` untuk memaksa Node 20.

---

2) Struktur Monorepo

```
hris/
  apps/
    web/           # Next.js 14 (App Router)
    api/           # NestJS 10
  packages/
    ui/            # (opsional) komponen bersama
    config/        # eslint/tsconfig share
```

```
    types/          # tipe TypeScript share
  infra/
    compose/        # docker-compose dev services
    k8s/            # manifest/helm (nanti)
  .nvmrc
  pnpm-workspace.yaml
  .editorconfig
  .gitignore
```

pnpm-workspace.yaml

```
packages:
  - 'apps/*'
  - 'packages/*'
```

---

3) Bootstrap Frontend (Next.js 14.2.x)

3.1 Inisialisasi app

```
cd hris
pnpm dlx create-next-app@14.2.5 apps/web \
  --ts --eslint --tailwind --src-dir --app --import-alias "@/*" --use-pnpm
```

> Kalau `14.2.5` belum tersedia di `create-next-app`, jalankan dengan versi terbaru lalu **edit** `package.json` agar `next` = `14.2.5` atau `14.2.x`, lalu `pnpm i`.

3.2 Versi dependency (package.json)

Di `apps/web/package.json`, pastikan (contoh):

```
{
  "dependencies": {
    "next": "14.2.5",
    "react": "18.2.0",
    "react-dom": "18.2.0",
    "@tanstack/react-query": "^5.0.0",
    "react-hook-form": "^7.51.0",
    "zod": "^3.23.0",
    "recharts": "^2.10.0"
  },
  "devDependencies": {
```

```
    "tailwindcss": "^3.4.0",
    "postcss": "^8.4.0",
    "autoprefixer": "^10.4.0"
  }
}
```

Lalu **install**:

```
cd apps/web
pnpm i
```

3.3 Konfigurasi Tailwind + shadcn/ui

1) Tailwind sudah dibuat otomatis. Update `tailwind.config.ts` untuk shadcn:

```
import type { Config } from 'tailwindcss'
const config: Config = {
  darkMode: ['class'],
  content: [
    './src/**/*.{ts,tsx}',
    './components/**/*.{ts,tsx}',
    './app/**/*.{ts,tsx}'
  ],
  theme: { extend: {} },
  plugins: [require('tailwindcss-animate')]
}
export default config
```

2) Install shadcn CLI & base deps:

```
pnpm dlx shadcn-ui@latest init -d
pnpm i class-variance-authority clsx tailwind-merge tailwindcss-animate
```

3) Generate komponen dasar (Button, Input, Card, Dialog):

```
pnpm dlx shadcn-ui@latest add button input card dialog label toast dropdown-menu
```

3.4 Setup React Query (provider)

Buat `src/providers/query-client.tsx`:

```
'use client'
import { QueryClient, QueryClientProvider } from '@tanstack/react-query'
import { ReactNode, useState } from 'react'

export default function RQProvider({ children }: { children: ReactNode }) {
  const [client] = useState(() => new QueryClient())
  return <QueryClientProvider client={client}>{children}</QueryClientProvider>
}
```

Di `app/layout.tsx`:

```
import RQProvider from '@/providers/query-client'
export default function RootLayout({ children }: { children: React.ReactNode })
{
  return (
    <html lang="en">
      <body>
        <RQProvider>{children}</RQProvider>
      </body>
    </html>
  )
}
```

3.5 Env & script

Buat `apps/web/.env.local`:

```
NEXT_PUBLIC_API_BASE_URL=http://localhost:3001
NEXT_PUBLIC_KEYCLOAK_URL=http://localhost:8080
NEXT_PUBLIC_KEYCLOAK_REALM=hris-dev
NEXT_PUBLIC_KEYCLOAK_CLIENT_ID=web
```

Script dev: `pnpm dev` (port default: 3000).

---

4) Bootstrap Backend (NestJS 10)

4.1 Inisialisasi app

```
cd ../../apps
pnpm dlx @nestjs/cli@10 new api --package-manager pnpm
cd api
```

4.2 Dependency inti

```
pnpm add @nestjs/config @nestjs/swagger swagger-ui-express \
  class-validator class-transformer \
  @nestjs/throttler @nestjs/cache-manager ioredis \
  @nestjs/passport passport @nestjs/jwt passport-jwt bcrypt \
  @prisma/client
pnpm add -D prisma @types/bcrypt @types/express
```

4.3 Prisma + PostgreSQL 16

```
pnpm prisma init --datasource-provider postgresql
```

`prisma/schema.prisma` (subset awal):

```
generator client { provider = "prisma-client-js" }
datasource db { provider = "postgresql" url = env("DATABASE_URL") }

model Tenant {
  id        String  @id @default(uuid())
  name      String
  createdAt DateTime @default(now())
  entities  Entity[]
}

model Entity {
  id        String  @id @default(uuid())
  tenantId  String
  name      String
  timezone  String  @default("Asia/Jakarta")
  tenant    Tenant  @relation(fields: [tenantId], references: [id])
  employees Employee[]
}
```

```
model Employee {
  id         String  @id @default(uuid())
  tenantId   String
  entityId   String?
  employeeNo String
  firstName  String
  lastName   String?
  email      String? @unique
  hireDate   DateTime
  status     String
  createdAt  DateTime @default(now())
  updatedAt  DateTime @updatedAt
  @@unique([tenantId, employeeNo])
}
```

`.env` (apps/api):

```
DATABASE_URL=postgresql://postgres:postgres@localhost:5432/hris
REDIS_URL=redis://localhost:6379
PORT=3001
JWT_SECRET=dev_dev_dev
```

Lalu:

```
pnpm prisma migrate dev --name init
```

4.4 Module contoh (Employees)

```
pnpm dlx @nestjs/cli@10 g module employees
pnpm dlx @nestjs/cli@10 g service employees --no-spec
pnpm dlx @nestjs/cli@10 g controller employees --no-spec
```

`employees.controller.ts` (contoh minimal):

```
import { Controller, Get, Query } from '@nestjs/common'
import { EmployeesService } from './employees.service'

@Controller('employees')
export class EmployeesController {
  constructor(private readonly svc: EmployeesService) {}
  @Get()
  list(@Query('page') page = 1, @Query('page_size') pageSize = 50) {
```

```
    return this.svc.list({ page: +page, pageSize: +pageSize })
  }
}
```

`employees.service.ts` (contoh minimal):

```
import { Injectable } from '@nestjs/common'
import { PrismaClient } from '@prisma/client'
const prisma = new PrismaClient()

@Injectable()
export class EmployeesService {
  async list({ page, pageSize }: { page: number; pageSize: number }) {
    const [items, total] = await Promise.all([
      prisma.employee.findMany({ skip: (page-1)*pageSize, take: pageSize }),
      prisma.employee.count()
    ])
    return { total, items }
  }
}
```

`main.ts` (Swagger + validation):

```
import { NestFactory } from '@nestjs/core'
import { AppModule } from './app.module'
import { ValidationPipe } from '@nestjs/common'
import { DocumentBuilder, SwaggerModule } from '@nestjs/swagger'

async function bootstrap() {
  const app = await NestFactory.create(AppModule)
  app.useGlobalPipes(new ValidationPipe({ whitelist: true, transform: true }))
  const config = new DocumentBuilder().setTitle('HRIS
API').setVersion('v1').build()
  const doc = SwaggerModule.createDocument(app, config)
  SwaggerModule.setup('docs', app, doc)
  await app.listen(process.env.PORT ?? 3001)
}
bootstrap()
```

Run: `pnpm start:dev`.

---

5) Infrastruktur Dev (Docker Compose)

Buat `infra/compose/docker-compose.yml`:

```
services:
  postgres:
    image: postgres:16.3
    environment:
      POSTGRES_PASSWORD: postgres
      POSTGRES_DB: hris
    ports: ["5432:5432"]
    volumes: ["pgdata:/var/lib/postgresql/data"]

  redis:
    image: redis:7.2
    ports: ["6379:6379"]

  kafka:
    image: bitnami/kafka:3.7.0
    environment:
      - KAFKA_ENABLE_KRAFT=yes
      - KAFKA_CFG_NODE_ID=1
      - KAFKA_CFG_PROCESS_ROLES=controller,broker
      - KAFKA_CFG_CONTROLLER_QUORUM_VOTERS=1@localhost:9093
      -
KAFKA_LISTENER_SECURITY_PROTOCOL_MAP=CONTROLLER:PLAINTEXT,PLAINTEXT:PLAINTEXT
      - KAFKA_CFG_LISTENERS=PLAINTEXT://:9092,CONTROLLER://:9093
      - KAFKA_CFG_ADVERTISED_LISTENERS=PLAINTEXT://localhost:9092
    ports: ["9092:9092"]
    volumes: ["kafka:/bitnami/kafka"]

  keycloak:
    image: quay.io/keycloak/keycloak:24.0
    command: ["start-dev"]
    environment:
      - KEYCLOAK_ADMIN=admin
      - KEYCLOAK_ADMIN_PASSWORD=admin
    ports: ["8080:8080"]

volumes:
  pgdata:
  kafka:
```

Jalankan:

```
cd infra/compose
docker compose up -d
```

---

6) Single Sign-On (Keycloak 24.x) — Dev Quickstart

1. Akses **http://localhost:8080** → login `admin/admin`.
2. Buat **Realm**: `hris-dev`.
3. Buat **Client**: `web` (public)
4. Root URL: `http://localhost:3000`
5. Valid redirect: `http://localhost:3000/*`
6. Buat **Client**: `api` (confidential)
7. Service accounts ON; simpan secret untuk backend.
8. Buat **User**: `dev.user@example.com` (password set) → tambahkan role `employee`.

(Integrasi OIDC ke Next & Nest akan ditambah pada sprint auth. Untuk sekarang, variabel env sudah disiapkan.)

---

7) Wiring Frontend ↔ Backend

Di `apps/web/src/lib/api.ts` (helper fetch):

```
export const API = process.env.NEXT_PUBLIC_API_BASE_URL!
export async function get<T>(path: string) {
  const r = await fetch(`${API}${path}`, { cache: 'no-store' })
  if (!r.ok) throw new Error(await r.text())
  return (await r.json()) as T
}
```

Contoh page `app/employees/page.tsx`:

```
import { useQuery } from '@tanstack/react-query'
import { get } from '@/lib/api'

export default function EmployeesPage() {
  const { data, isLoading } = useQuery({ queryKey: ['employees'], queryFn: ()
=> get<{total:number;items:any[]}>('/employees') })
  if (isLoading) return <div>Loading...</div>
  return (
    <div>
      <h1 className="text-xl font-semibold">Employees</h1>
```

```
      <pre className="text-sm bg-muted p-4 rounded-lg overflow-
auto">{JSON.stringify(data, null, 2)}</pre>
    </div>
  )
}
```

---

8) Script Root & Quality Gates

package.json (root)

```
{
  "private": true,
  "scripts": {
    "dev:web": "pnpm --filter web dev",
    "dev:api": "pnpm --filter api start:dev",
    "dev:up": "docker compose -f infra/compose/docker-compose.yml up -d",
    "dev:down": "docker compose -f infra/compose/docker-compose.yml down -v",
    "lint": "pnpm -r lint",
    "typecheck": "pnpm -r typecheck"
  }
}
```

.editorconfig (root):

```
root = true
[*]
indent_style = space
indent_size = 2
end_of_line = lf
charset = utf-8
trim_trailing_whitespace = true
insert_final_newline = true
```

---

9) Testing & Tools (sesuai SRS)

- **Web**: Playwright (E2E) → `pnpm dlx playwright@1.46 install` (di `apps/web`).
- **API**: Jest + Supertest sudah default di Nest; tambahkan sample test health.
- **Load**: `k6` (local, opsional di fase awal).
- **Lint/Type**: ESLint + TypeScript strict.

Contoh health controller (API):

```
pnpm dlx @nestjs/cli@10 g controller health --no-spec
```

`health.controller.ts`:

```
import { Controller, Get } from '@nestjs/common'
@Controller('health')
export class HealthController { @Get() ok() { return { status: 'ok' } } }
```

---

10) Jalankan End-to-End (dev)

```
# 1) Service infra
pnpm dev:up

# 2) API
cd apps/api
pnpm start:dev

# 3) DB migrate
pnpm prisma migrate dev

# 4) Web
cd ../../apps/web
pnpm dev
```

Buka **http://localhost:3000/employees** → harus tampil `{ total, items: [] }` (kosong karena belum seed).

---

11) Seed Data Dev (opsional cepat)

Buat `apps/api/prisma/seed.ts`:

```
import { PrismaClient } from '@prisma/client'
const db = new PrismaClient()
async function main() {
  const t = await db.tenant.upsert({ where: { id:
'00000000-0000-0000-0000-000000000001' }, update: {}, create: { id:
'00000000-0000-0000-0000-000000000001', name: 'DevTenant' } })
```

```
  const e = await db.entity.create({ data: { tenantId: t.id, name:
'DevEntity' } })
  await db.employee.create({ data: { tenantId: t.id, entityId: e.id,
employeeNo: 'E-0001', firstName: 'Budi', hireDate: new Date(), status:
'ACTIVE' } })
}
main().finally(() => db.$disconnect())
```

`package.json` (apps/api):

```
{
  "scripts": { "seed": "ts-node prisma/seed.ts" },
  "devDependencies": { "ts-node": "^10.9.2" }
}
```

Jalankan: `pnpm prisma db seed` **atau** `pnpm seed` sesuai preferensi.

---

12) Checklist “Done Right”

- [ ] Node 20 aktif (`node -v` → v20.x)
- [ ] `docker compose ps` menampilkan postgres/redis/kafka/keycloak up
- [ ] `http://localhost:3001/docs` (Swagger) terbuka
- [ ] `http://localhost:3000` (Next) jalan
- [ ] `/employees` mengembalikan data

---

13) Next Steps (lanjutan sprint)

- Tambah **RLS** & kolom `tenant_id` di semua tabel; middleware Prisma untuk inject scope.
- Implement **JWT** (Keycloak resource server) di Nest.
- Buat **gateway attendance** (service kecil) → publish event ke Kafka.
- Setup **Playwright** E2E minimal (login flow dummy).
- Tambahkan **GitHub Actions** (lint, typecheck, test, build).

---

14) Troubleshooting Cepat

- **Next tidak start** → cek Node 20 & dep konflik; hapus `node_modules` + `pnpm store prune` + `pnpm i`.
- **API tidak konek DB** → cek `DATABASE_URL` & apakah Postgres up; `docker logs`.
- **Kafka client gagal connect** → pastikan `KAFKA_CFG_ADVERTISED_LISTENERS` sesuai host lokal.

- **Keycloak 404** → realm belum dibuat; cek URL `/admin/` untuk login admin dulu.

Selesai. Tinggal kamu ikuti dari atas ke bawah; kalau macet di titik tertentu, lihat bagian Troubleshooting. Kita jaga **Next 14 / Nest 10 / Postgres 16**—nggak naik turun versi liar.